

Requirement Analysis and Design Document

Astrallog-HPC

High-Performance Computing Space Mission Logging System

PREPARED BY

Luca Salmoiraghi
Tobia Grimaldi
Dong Hua Zhu

PROJECT SCOPE

This document outlines the domain assumptions, functional and non-functional requirements, system architecture, and design patterns for the Astralog-HPC environment.

DATE OF CREATION

April 4, 2026

LAST UPDATED ON

June 29, 2026

Contents

1	Introduction to the project and project goals	2
1.1	Overview	2
1.2	Goals	2
2	Requirement analysis	3
2.1	Relevant human and non-human actors	3
2.1.1	Human Actors	3
2.1.2	Non-Human Actors	3
2.2	Use cases	3
2.3	Domain assumptions	5
2.4	Requirements	5
2.4.1	Functional requirements	5
2.4.2	Non-functional requirements	6
3	Design	8
3.1	General description of the architecture	8
3.1.1	Pipeline	8
3.1.2	Design patterns	8
3.2	Class Diagram	10
3.2.1	Simplifications and variations	11
3.3	Sequence diagrams	11
3.3.1	Scenario 1: Accumulation	11
3.3.2	Scenario 2: Rule Loading	11
3.3.3	Scenario 3: Rule Processing	11
3.4	Critical points and design decisions	14
3.4.1	Architectural decisions	14
3.4.2	Implementation and HPC integration	14
3.4.3	Quality assurance (CI/CD)	14
4	Work distribution and effort	15
5	On the usage of AI	16

1 Introduction to the project and project goals

1.1 Overview

Modern space missions produce a large and uninterrupted flow of telemetry data. To safeguard spacecraft, the development of **AstraLog-HPC** is required. AstraLog-HPC is a software tool designed for anomaly detection in onboard systems. It ingests data streaming from spacecraft sensors within each system and subsystem, filters systematically invalid packets and applies monitoring rules based both on the absolute value of incoming data as well as on their changes over time and their interdependencies. AstraLog-HPC is designed to produce both a summary of non-anomalous telemetry and a log of detected errors for mission personnel, enabling them to prioritise safety decisions and plan subsequent actions that safeguard mission success. Additionally, the system must support operation both onboard, on embedded systems for real-time ingestion and high-priority anomaly detection, and on-ground in data centres, for high-volume, lower-priority data analysis.

This project aims to put into practice what was learned from the first part of the Software Engineering for HPC course. In particular, this document covers principles of requirement engineering and architectural design.

1.2 Goals

The primary goals of AstraLog-HPC can be organised into three categories:

- **Robust Data Ingestion and Cleansing:** The system must be capable of receiving a continuous stream of JSON telemetry data from various sensors across different subsystems. It shall detect and discard invalid packets (further discussed in Table 3) and noisy measurements commonly occurring in real-time space communication.
- **Collect Data into Batches for Processing:** Valid packets shall be accumulated into a local batch file. When a batch reaches a specified size and/or enough time from the last sent batch is elapsed, the system triggers the Anomaly Detection phase.
- **Anomaly Detection and Rules Application:** The core goal of the system is to analyse valid packets of data and detect anomalies according to four different types of rules (further discussed in Table 4).

2 Requirement analysis

2.1 Relevant human and non-human actors

This section identifies the external entities that interact with AstraLog-HPC. Actors are categorized based on whether they directly interact with the system (Primary) or provide support to achieve a goal (Secondary). Table 1 provides an overview of identifiable actors.

Actor	Category	Responsibility
Ground Station Operator	Primary (Human)	Responsible for spacecraft configuration and health monitoring. They define the environment via configuration files and serve as the primary consumer of system alerts.
MQTT Broker	Primary (Non-Human)	Acts as the central message intermediary. The system interacts with the broker subscribing on a topic in order to receive the stream of packets.
Sensors	Secondary (Non-Human)	Hardware or software components on the aircraft that function as data producers. They capture telemetry data, encapsulate it into JSON packets, and publish it to the Broker.
SLURM Job Scheduler	Secondary (Non-Human)	Enables access to the HPC cluster. Responsible for allocating resources and scheduling the deployment of system components.

Table 1: AstraLog-HPC System Actors and Responsibilities

2.1.1 Human Actors

The main human actor that interacts with the system is:

- **Ground Station Operator (Primary):** This actor is responsible for the configuration and health monitoring of the spacecraft. The operator defines the environment by providing the configuration files which specify the rules to be aware of and the sensors sending data. Furthermore, operators serve as the primary consumer of system alerts, accessing `alarms.log` to make high-level safety decisions regarding aircraft subsystems.

2.1.2 Non-Human Actors

Main non human actors that interact with the system are:

- **MQTT Broker (Primary):** First component the system interacts with in order to receive the packets stream from spacecraft. AstraLog-HPC establishes a permanent connection with the Broker subscribing to the topic `esa/astralogs/telemetry`.
- **Sensors (Secondary):** Hardware or Software entities that reside on the spacecraft. They are responsible for all the measurements regarding the spacecraft status (e.g. temperature, voltage, current, pressure). They do not directly interact with the AstraLog-HPC environment and are therefore reported here only for the sake of completeness.
- **SLURM Job Scheduler (Secondary):** This entity manages access to an HPC cluster. It is responsible for allocating resources and scheduling the deployment of AstraLog-HPC components when the system runs in ground-based, high-volume processing mode.

2.2 Use cases

This section describes the main use cases of the AstraLog-HPC system. The use cases define the functionalities provided by the application as well as the interactions between them and the actors that interact with the system. These interactions are visualised in the use case diagram below (Figure 1), while Table 2 provides a terse overview of the role of each actor in each use case.

Description of the Presented Use Cases

- **IngestData:** The system receives a continuous stream of JSON packets from the aircraft sensors. This process includes the mandatory validation of packet syntax and structure.
- **DetectCorruptionCases:** Triggered during ingestion, the system filters out invalid packets. It specifically identifies malformed JSON, schema errors (missing mandatory fields), and type errors (invalid data formats).
- **MonitorSystemStatus:** The system aggregates valid packets in batches and later evaluates the state of the spacecraft given a set of monitoring rules in `rules.json`.

Conditional Outcomes

The following use cases extend *MonitorSystemStatus*, defining the behaviour of the system if an aggregated batch respects or not one or more of the monitoring rules defined in `rules.json`:

- **LogNominalState:** If a data batch is evaluated and no violation is detected, the system appends an entry to the `valid_data.csv` file.
- **ReportAlarm:** If one or more rule is evaluated to TRUE, the incorrect state of the system is reported with an alarm in the `alarm.log`.

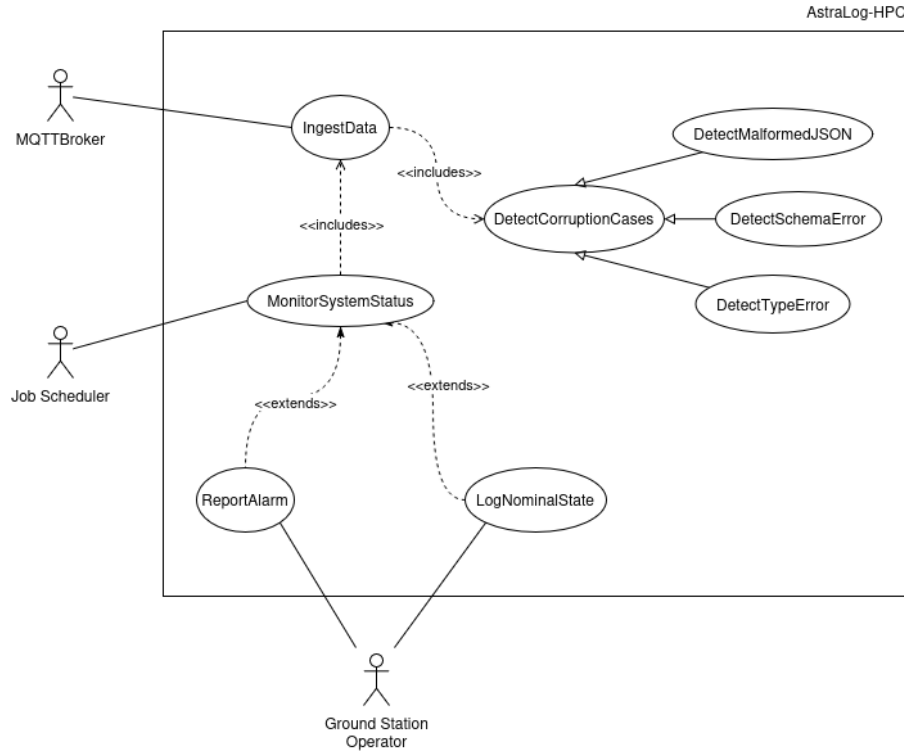


Figure 1: AstraLog-HPC Use Case Diagram

Use Case	Primary Actor	Secondary Actor
IngestData	MQTT Broker	Sensors
DetectInvalidPackets	(internal)	None
MonitorSystemStatus	Ground Station Operator	SLURM (for HPC mode)
LogNominalState	Ground Station Operator	None
ReportAlarm	Ground Station Operator	None

Table 2: Use Case to Actor Mapping

2.3 Domain assumptions

This section documents the domain-specific assumptions underlying and justifying the design decisions of AstraLog-HPC. These assumptions are accepted as true and the software is developed according to these premises. The correct behaviour of the system is not guaranteed outside of these assumptions.

Specifically, the system assumes the following:

- The spacecraft systems and subsystems operate over a unified synchronous system clock.
- All measurements across all sensors are equally spaced in time and perfectly aligned (that is, all sensors produce data for the entire duration of the mission, no failures of sensors). Each sensor publishes on a specific topic with a 1Hz rate.
- Each spacecraft contains multiple sensors types, each eventually available in multiple instances. The type of sensor is determined by the particular quantity which is measured.
- The specific type and number of sensors to be managed are available in a configuration file called **sensors.yaml**, which means the number of sensors is assumed to remain constant throughout the entire operation of the system.
- Each sensor continuously produces data, which is published to the MQTT Broker in the form of a standardised **.json** package with the following example structure:

```
{
  "timestamp": "2025-11-15T12:00:00Z",
  "sensor_id": "TEMP-01",
  "value": 25.5,
  "priority": "HIGH"
}
```

- Data transmission is often noisy, which means received packets could be invalid, having missing data or wrong format.
- The data collected in a certain clock cycle, corresponds to the n -th measurement of each sensor. This means all the packets received together belong to measurements happened at the same time.
- Monitoring rules are available in a **rules.json** file, accessible by AstraLog-HPC.
- Each monitoring rule in the previously mentioned **rules.json** file belongs to one of four types of rules (Table 4) Moreover, each monitoring rule has a unique ID.
- Monitoring rules check is performed just after the system collects a batch size of valid packets from the Broker.
- Eventually, monitoring rules are characterized by a *PRIORITY* attribute. Rules with an higher priority parameter are evaluated before the others among the same type.

2.4 Requirements

This section specifies the functional and non-functional requirements for AstraLog-HPC. Functional requirements specify what the system must do, while non-functional requirements define quality constraints and shape how the system performs its actions.

2.4.1 Functional requirements

The following functional requirements describe the specific behaviours, tasks and operations that AstraLog-HPC must support:

- The system should receive the streamed data in the format of JSON packets by establishing a persistent subscription to the MQTT Broker on the **esa/astralog/telemetry** topic.
- The system should filter invalid packets. A packet is considered invalid whenever one of the following corruption cases verifies (Table 3):
- The system should accumulate the valid packets in a batch file to be used to analyse data according to the monitoring rules (Table 4):

Error Type	Description
Malformed JSON	Transmitted packet has an invalid JSON syntax.
Schema Errors	A packet is missing mandatory fields required for processing.
Type Errors	A sensor is sending data of an invalid or unexpected data type.

Table 3: Definition of Packet Corruption Cases

Rule Type	Evaluation Logic	Trigger Condition
Simple Rule	Absolute Threshold	Evaluates if a sensor measurement crosses a fixed threshold at a specific point in time.
Step Difference	Relative Variation	Evaluates the difference between the current measurement and the immediately preceding one.
Stateful Rule	Persistence	Triggers only when a condition is continuously violated over a defined number of consecutive measurements.
Logical Correlation	Boolean Combination	Combines outcomes of multiple rules at the same timestamp using operators like AND or OR.

Table 4: AstraLog-HPC Monitoring Rule Definitions

In particular, after having filtered the invalid packets and sent them to the ground station, the system should operate in the following way:

- if no rules are violated, the system should append a single aggregated line to the output file `valid_data.csv` following the format:

```

TIMESTAMP ; NOMINAL ; [SENSOR_1] : [VALUE_1] | [SENSOR_2] : [VALUE_2] | ...

```

An example is provided:

```

2025-11-15T12:00:00Z ; NOMINAL ; TEMP-01:25.5 | PRES-01:101.3 | VOLT-MAIN:24.1

```

- if one or more rules are violated, nothing is written to `valid_data.csv`, while each violated rule is reported in the output file `alarms.log` following the format:

```

TIMESTAMP ; RULE_ID ; PRIORITY ; VIOLATED_SENSOR(S) ; CURRENT_VALUE(S)

```

Two examples are provided:

```

2025-11-15T12:00:05Z ; R1 ; MEDIUM ; TEMP-01 ; 51.2
2025-11-15T12:00:05Z ; R4 ; HIGH ; TEMP-01 , PRES-01 ; 51.2 , 98.0

```

2.4.2 Non-functional requirements

The following non-functional requirements define the quality characteristics and constraints of AstraLog-HPC, including performance, reliability, scalability, and operational boundaries:

- In order to handle the continuous flow of data from a spacecraft, AstraLog-HPC must be optimised for batch processing to handle high-throughput sensor data efficiently.
- The system should maintain data consistency by subscribing to an *HiveMQ MQTT* Broker. This means that the entire telemetry must be collected on a topic and maintained safe even in conditions of network instability.
- The system must be capable of executing with a consistent behaviour in multiple environments, from classical to virtual machines and HPC clusters.
- The deployed architecture must allow access to an HPC cluster through a SLURM Job Scheduler.
- To exploit the HPC cluster capabilities, the system should be capable of optimising data ingestion aggregating JSON packets into batches.
- The system should support parallel evaluation of rule across data batches to fully exploit the capabilities of an HPC environment.

- Given the highly critical field of application of the software, reliability and stability are crucial characteristics the system should pursue.
- The system must produce coherent output text in order to permit verifiability of software and allowing to correctly perform safety checks on the implementation.
- The software structure must be clearly defined and documented, allowing for the easy addition of new rule types and eventual newly defined behaviours.

3 Design

3.1 General description of the architecture

The AstraLog software base relies on a **Component-Based Architecture**. Each component of the system performs a specific, well-defined job and interacts with the others exclusively through interfaces, ensuring reliability, scalability and maintainability.

The system is comprised of the following components:

- **DataIngestor**: handles the stream of data provided by the Broker and imports it in the Astralog-HPC system; It also filters out syntactically invalid packets, given the corruption cases already defined (Table 3).
- **BatchAccumulator**: accumulates filtered (hence valid) packets in a local batch file for further processing, it must accumulate up to a specified batch file dimension;
- **RuleLoader**: reads and provides access to static configurations and rules, specifically monitoring rules from `rules.json`. Moreover, it handles the reordering of these rules given a *PRIORITY* parameter;
- **ThreadSafeBuffer**: collects the complete batches accumulated in the BatchAccumulator and appends them in a queue structure;
- **RuleEngine**: pops the first element from the ThreadSafeBuffer's queue and applies the previously loaded rules to the batch according to a priority value (low, medium, high);
- **MeasDatabase**: database where we store previous measurements associated to each sensor. Specifically used to evaluate step-difference and stateful rules;
- **OutputDispatcher**: directs the evaluation results to the correct files. If no rules are violated, the batch files is appended to `valid_data.csv`, otherwise nothing is written to `valid_data.csv`, while all the violated rules are reported on `alarms.log`.

Their mutual relation is expressed by the following component diagram (Figure 2):

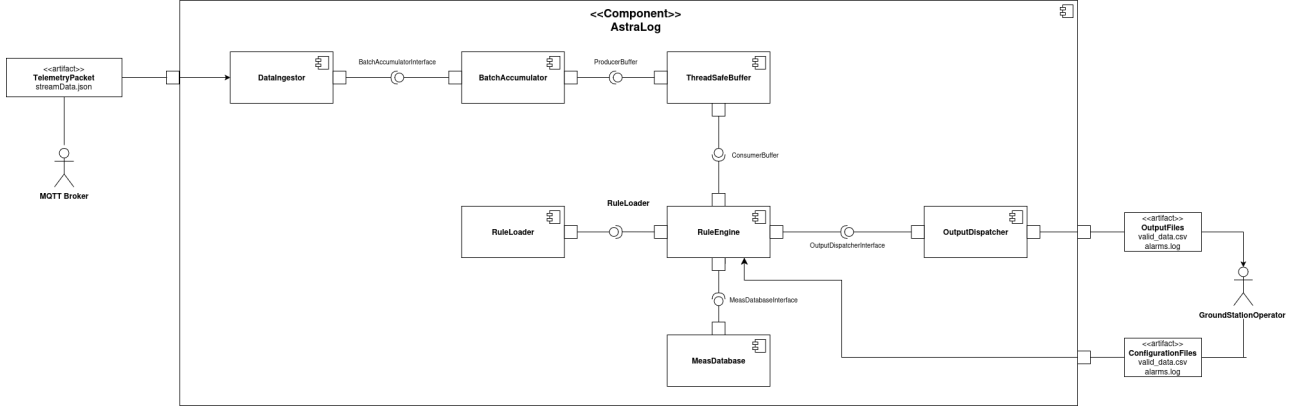


Figure 2: AstraLog-HPC: Component Diagram

3.1.1 Pipeline

The data flow follows a clear pipeline: DataIngestor receives raw stream data from external sources, filters out malformed JSON entries, and parses the valid ones. The parsed data is forwarded to BatchAccumulator, which buffers incoming records until a configured batch size is reached, then pushes the batch into ThreadSafeBuffer. The RuleEngine consumes batches from the buffer, applies the configured rules, and sends a result summary to OutputDispatcher.

3.1.2 Design patterns

To structure the pipeline, three design patterns are employed:

- **Producer-Consumer**: BatchAccumulator acts as the producer, pushing batches into the ThreadSafeBuffer queue, while RuleEngine acts as the consumer, extracting and processing them. This decouples data accumulation from rule processing and ensures thread-safe communication between the two;

- **Strategy:** RuleEngine acts as the context and operates on a BaseRule abstract class. Each concrete rule is a subclass of BaseRule, allowing different rule implementations to be selected and applied interchangeably at runtime without modifying the engine logic;
- **Centralised Rule Creation:** RuleLoader is responsible for instantiating the correct BaseRule subclass based on the rule configuration, centralising and decoupling object creation from the rest of the system.

3.2 Class Diagram

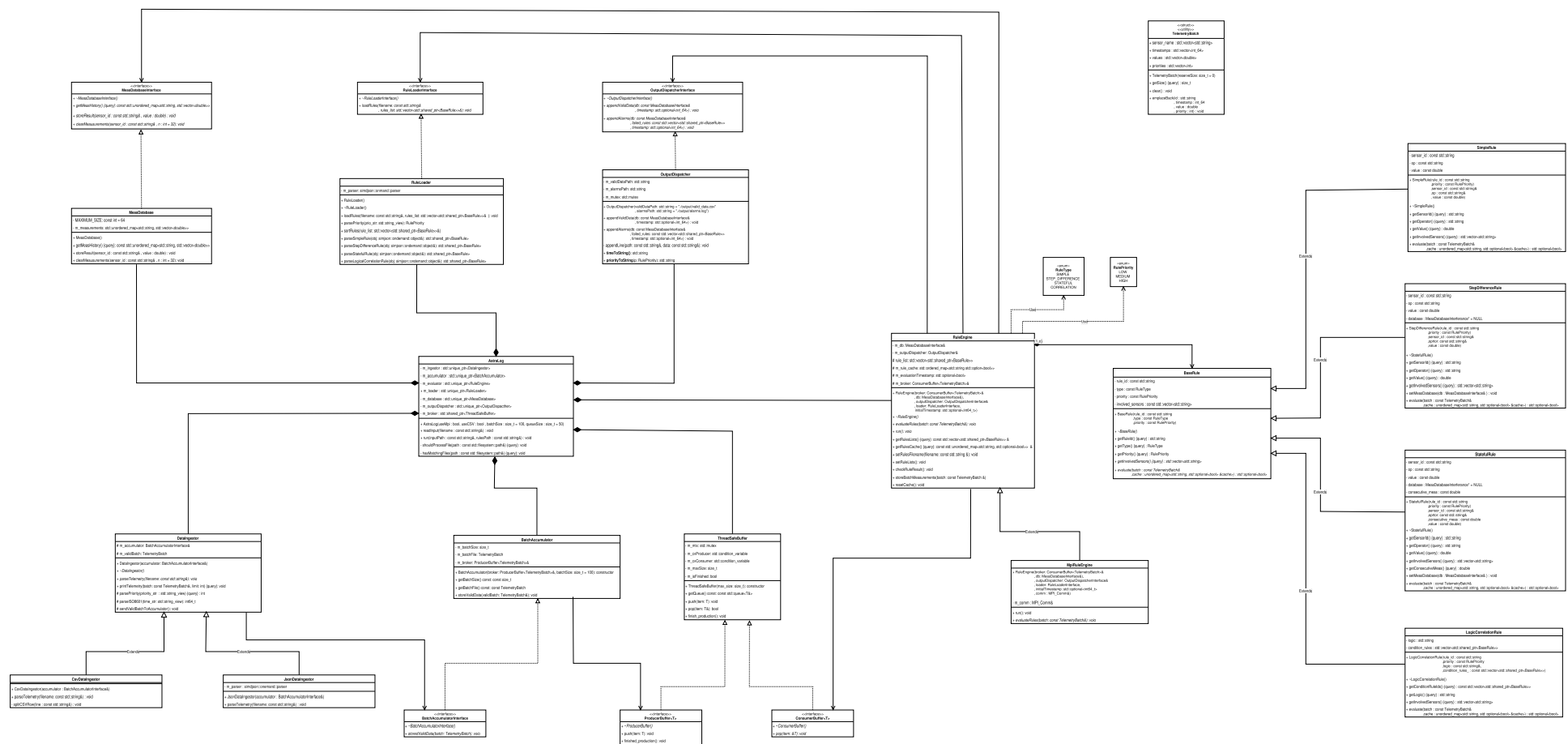


Figure 3: AstraLog-HPC: Class Diagram

3.2.1 Simplifications and variations

The RuleLoader component was originally conceived following the Factory Method pattern, where separate creator subclasses would each be responsible for instantiating a specific BaseRule subtype.

In the actual implementation, this was simplified into a Simple Factory approach: a single loadRules method dispatches object creation through an if/elseif chain based on the rule "type" field, with dedicated private parsing methods (parseSimpleRule, parseStepDifferenceRule, parseStatefulRule, parseLogicalCorrelationRule) for each concrete type. While this does not constitute a formal GoF pattern, it preserves the core intent of centralising and decoupling rule instantiation from the rest of the system.

3.3 Sequence diagrams

A sequence diagram shows how objects interact with each other over time in a system. It helps visualise the order of messages and operations between objects.

The following sequence diagrams correspond to different scenarios that highlight the main functionalities of the system.

3.3.1 Scenario 1: Accumulation

This scenario (Figure 4) illustrates the data accumulation pipeline, detailing how raw telemetry is validated and batched for downstream processing. The execution flow proceeds as follows:

- **Data Ingestion and Validation:** During the telemetry parsing phase, the DataIngestor validates the format of the raw incoming data, actively filtering out any packets that do not adhere to the communication protocol.
- **Batch Transfer:** Once validated, the ingestor bundles the clean data and forwards it to the BatchAccumulator via the storeValidData() method.
- **Iterative Accumulation:** The accumulator loops through the incoming payload. It extracts each individual measurement (accumulateSingleMeas()) and appends it to its own internal batch.
- **Threshold Trigger and Queueing:** Each time the internal batch reaches a predefined size threshold, the accumulator pushes the completed batch into the ThreadSafeBuffer queue, making it available for the rule engine to consume
- **Reset and Continue:** Following a successful push, the accumulator immediately clears its internal batch (clearBatch()) and resumes processing the remainder of the incoming data until the entire input batch has been read.

3.3.2 Scenario 2: Rule Loading

Before execution, the Rule Engine must load the rules required to evaluate data batches. The following scenario (Figure 5) shows the pipeline:

3.3.3 Scenario 3: Rule Processing

This scenario (Figure 6) shows how batches are processed from the queue.

- **Retrieval and Evaluation:** The Rule Engine pops a batch from the ThreadSafeBuffer and evaluates it against configured rules, storing intermediate results in the MeasDatabase
- **Dispatch Routing:** After checking the rule outcomes, the engine routes the data to the OutputDispatcher. If all rules pass, valid data is appended; if any fail, alarms are dispatched instead.
- **Cleanup and Residuals:** Finally, the engine resets its cache and, if any residual data remains in the sub-batch, performs a final evaluation and storage pass.

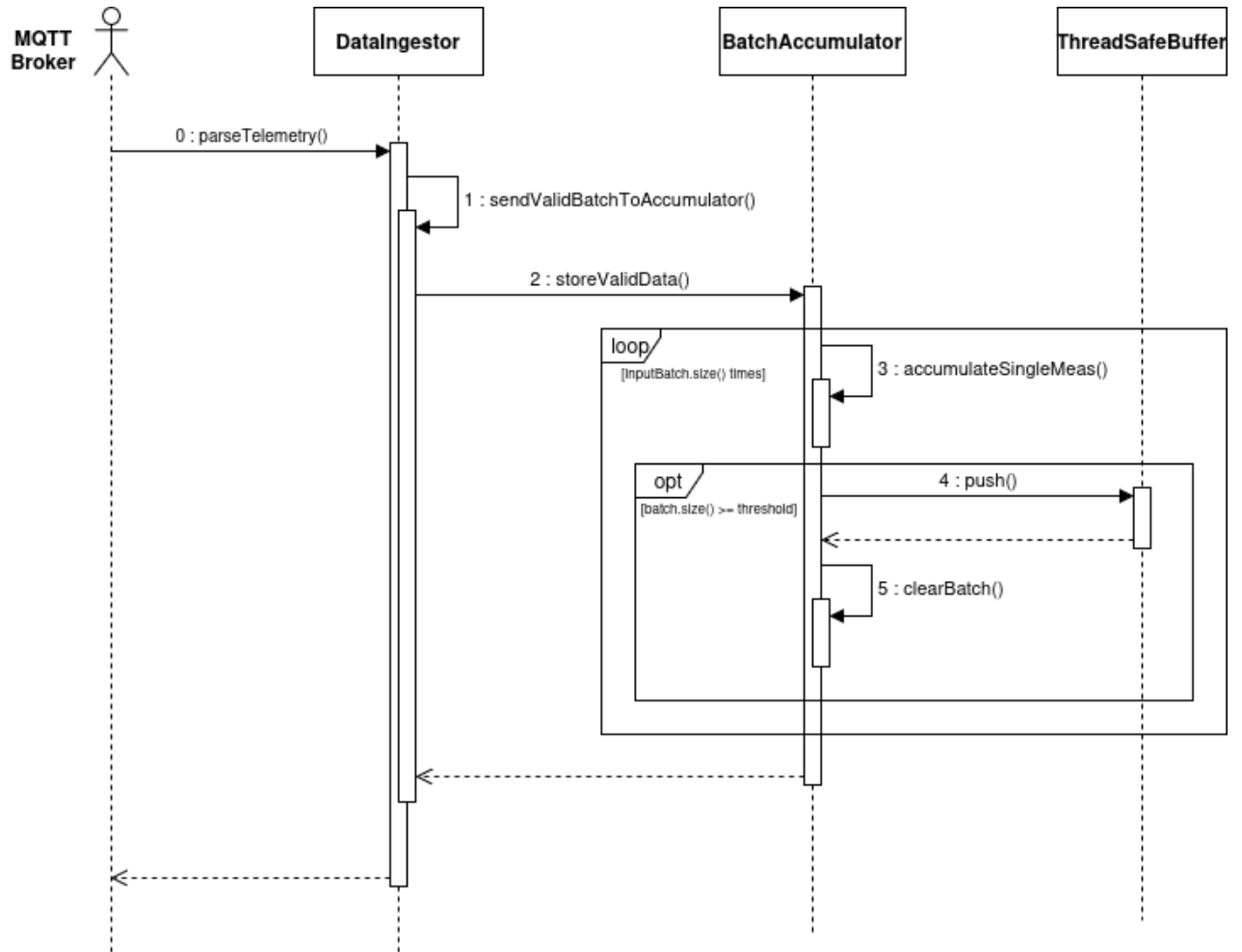


Figure 4: AstraLog-HPC: Accumulation - Sequence Diagram

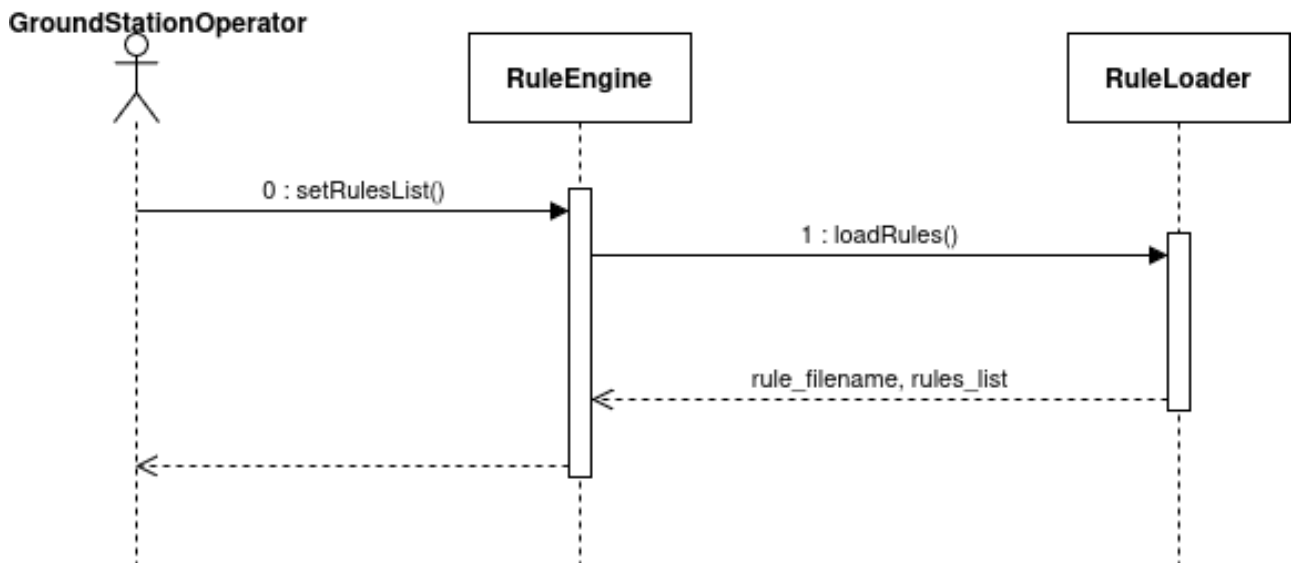


Figure 5: AstraLog-HPC: Rule Loading - Sequence Diagram

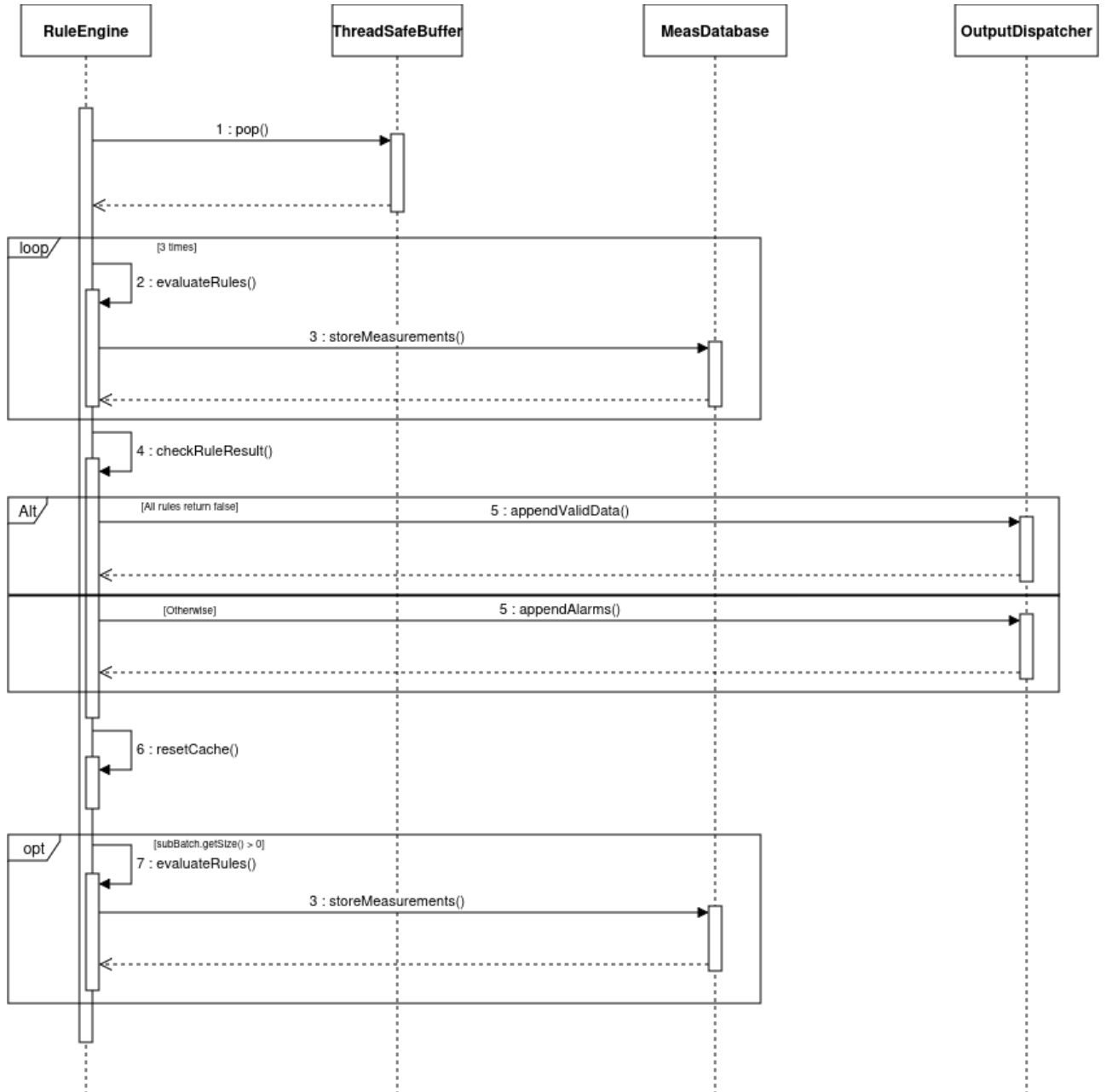


Figure 6: AstraLog-HPC: Rule Processing - Sequence Diagram

3.4 Critical points and design decisions

3.4.1 Architectural decisions

The following decisions regard the choices made to ensure the AstraLog-HPC environment can handle continuous data telemetry while maintaining data integrity:

- The system adopts an Event-Driven Architecture (**EDA**) using a message broker in order to collect JSON packets coming from spacecraft's sensor. With this approach the system is able to manage the stream of data coming from the aircraft in an asynchronous way. The MQTT Broker topic acts as a collector guaranteeing zero data loss.
- The filtering of packets received by the system is performed with a parsing function inside the **DataIngestor** component that evaluates the correctness of JSON syntax.
- **RuleEngine** needs to maintain the state of previous measurements of sensors in order to check the validity of *STATEFUL* rules. To do so, it is required a structure that acts as a memory and maps each *sensor_id* to the past measurements. This idea was later formalized into the MeasDatabase component.
- The design supports dual-mode operation: embedded execution for real time onboard ingestion and HPC cluster execution via SLURM for high-volume data analysis.

3.4.2 Implementation and HPC integration

The following points outline the technical strategies and patterns used to leverage High-Performance Computing capabilities for parallel data processing and efficient resource allocation:

- Data ingestion is managed via a Python script that utilizes a HiveMQ broker to receive a continuous data stream from an external academic website.
- The system is implemented adopting the **C++** programming language to meet low-latency requirements and utilize high-quality, parallel JSON parsers.
- To guarantee consistent deployment across the Cineca Galileo cluster, the application is containerised using Singularity, ensuring the execution environment does not vary from local testing to production cluster nodes.
- The system generates standardised `valid_data.csv` and `alarms.log` according to the previously reported format, ensuring that all safety checks are human-readable and verifiable by mission personnel.

3.4.3 Quality assurance (CI/CD)

To maintain the reliability and stability required for critical space mission software, the system incorporates automated workflows for continuous verification and validated output generation. In particular, the following strategies are adopted:

- A **CI/CD pipeline** is integrated via GitHub Actions, ensuring new software commit is automatically tested to verify stability and rule logic before deployment.

4 Work distribution and effort

The following table (Table 5) summarises the distribution of tasks for this document across team members and the estimated effort for each activity.

Task	Responsible	Effort (hours)	Status
Requirements elicitation	Tobia Grimaldi, Dong Hua Zhu	6	Completed
Use case analysis	Tobia Grimaldi	4	Completed
Domain assumptions	Luca Salmoiraghi	1	Completed
Functional requirements	Luca Salmoiraghi, Tobia Grimaldi	5	Completed
Non-functional requirements	Luca Salmoiraghi, Tobia Grimaldi	4	Completed
Component Diagram	Luca Salmoiraghi, Dong Hua Zhu	8	Completed
Class Diagram	Dong Hua Zhu	2	Completed
Sequence Diagrams	Dong Hua Zhu	8	Completed
Critical points and decisions	Tobia Grimaldi	1	Completed
Document formatting & LaTeX	Luca Salmoiraghi	5	Completed
AI tool integration & disclosure	Luca Salmoiraghi, Dong Hua Zhu, Tobia Grimaldi	1	Completed
Total		45	

Table 5: Work distribution and estimated effort

5 On the usage of AI

The following section details how Generative AI has been employed as a support tool for the project. In particular, the following actions have been AI-assisted:

- Grammatical review and general revision of sentence correctness (chatbot employed: DeepSeek);
- Title page styling (chatbot employed: Google Gemini), although it is to be noted that the final result is actually human-created with some suggestions integrated from the AI;
- Automation of table creation (chatbot employed: Google Gemini, DeepSeek), $\text{\LaTeX}$ code was generated starting from a human-readable description of each table.
- Code refactoring (chatbots employed: Google Gemini and Claude): Generative AI was utilized to draft useless lines of code and troubleshoot compilation errors. All AI-generated snippets were subsequently reviewed, refactored, and tested.
- Test cases generation (chatbot employed: Gemini): AI tools helped with providing a right set of test cases that were used in order to assess the functionalities of the system.

LLMs were largely used to review code quality and assist with architectural choices. These are some examples of the prompts used:

- Is the function `<function-name>` consistent? Please review its code quality and suggest improvements.
- What is the most efficient way to use MPI communicators for this specific function?
- Can you review this c++ code and tell if its structure is correct?

We used AI to accelerate testing and performance assessment. Some scripts were used to generate deterministic rules and test scenarios.

In particular, we used AI models (Gemini, Claude) to generate scripts that tested our model parallelisation performance. This allowed rapid and repeatable comparisons between the OpenMP and OpenMP+MPI variants and helped identify that MPI communication overhead dominated the execution time. AI-driven test generation and result aggregation significantly sped up our benchmarking iterations.

- Generate a script in c++ that tests the parallelization workflow. I want a deterministic set of rules and measurements that the system has to evaluate with both OpenMP and OpenMPI+OpenMP application. Provide also a bash script I can use to test different OpenMPI parallel distributions. Append the results in a .csv file.

In addition to that, AI was largely used in order to understand the constraints of Galileo100 cluster architecture. With the joint usage of Cineca's documentations and LLM we were able to figure out a CD pipeline able to deploy the application on an HPC environment with many limitations. Some examples:

- How can I configure a CI/CD pull-based workflow on the Galileo100 cluster?
- Is Galileo100 cluster allowing api calls to the Github API?
- Is it possible to run consecutive slurm jobs after a certain period of time given the cluster's crontab limitations?